Assignment 1 — GAN Playground

Deep Neural Networks for Computer Graphics

Released: May 14, 2026
Due: June 4, 2026, 23:59

Introduction

In this assignment you will implement, train, and analyze two GAN [1] architectures: a **Deep Convolutional GAN (DCGAN)** [2] for unconditional cat image generation, and a **CycleGAN** [4] for unpaired image-to-image translation between cat breeds. You will also integrate **Weights and Biases (W&B)** throughout to log and compare training runs.

The theory behind both models has been covered in lecture. The goal here is to translate that theory into working code and to build intuition through careful experimentation.

Skeleton Files

Download the assignment skeleton from the course portal. Files marked *provided* should not be modified.

File	Role	Description
`models.py`	Fill in TODOs	Architecture definitions
`vanilla_gan.py`	Fill in TODOs	DCGAN training loop
`cycle_gan.py`	Fill in TODOs	CycleGAN training loop
`data_loader.py`	Fill in TODOs	Dataset and data transforms
`diff_augment.py`	Provided	Differentiable augmentation
`utils.py`	Provided	Helper utilities

Environment Setup

Google Colab

We provide `run_colab.ipynb` for running the assignment in Google Colab. The notebook handles environment setup, dataset download, and launching training commands.

Before opening the notebook, upload the assignment skeleton folder to your Google Drive. This folder will serve as your working directory throughout the assignment. You will later need to update the assignment path inside the notebook so Colab can access your files.

Open the notebook in Colab and enable a GPU runtime via *Runtime* → *Change runtime type* → *T4 GPU*.

Important:

- When editing the skeleton `.py` files directly in the Colab file browser, make sure your changes are saved back to Drive.
- There is no need to submit the notebook itself.

Note: The notebook downloads the dataset automatically from **Hugging Face**, a popular platform for sharing machine learning models and datasets. If you are not already familiar with it, it is worth exploring – you will likely encounter it often in future projects.

Weights and Biases Setup

W&B (Weights and Biases) is an experiment tracking platform. Throughout this assignment, you will use it to log training losses, generated image grids, hyperparameters, and experimental comparisons. Later in the assignment, you will also create W&B Reports to organize and present your experimental results.

If you do not already have a W&B account, create a free account at `https://wandb.ai`

Next, generate an API key from your W&B account settings. You will need it later to authenticate when running experiments using the provided notebook.

The notebook will prompt you to log in using `wandb login`.

1. Part 1: Deep Convolutional GAN

For the first part of this assignment, we will implement a version of Deep Convolutional GAN (DCGAN) for unconditional image generation. The model uses a convolutional neural network as the discriminator, and a generator composed of upsampling and convolutional layers.

To implement the DCGAN, we will build three main components: (1) the generator, (2) the discriminator, and (3) the training procedure. We will develop each of these components in the following subsections.

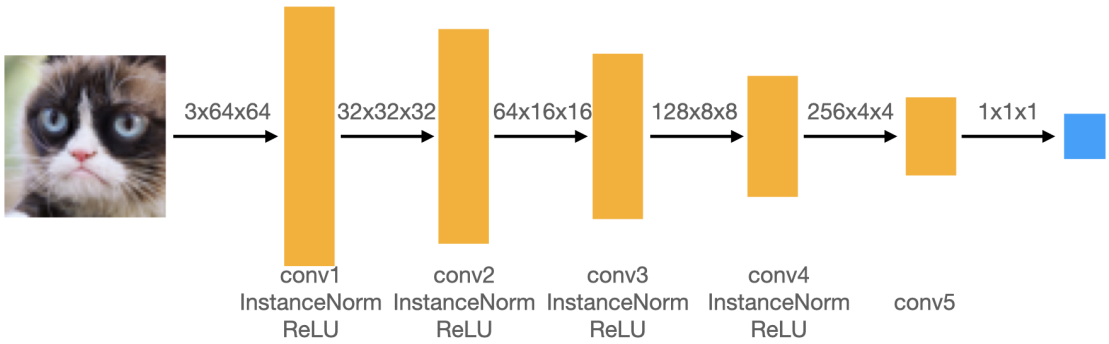
1.1. Data Augmentation

DCGAN will perform poorly without data augmentation on a small-sized dataset because the discriminator can easily overfit to a real dataset. To rescue, we need to add some data augmentation such as random crop and random horizontal flip.

Task 1.1. Fill in the `vanilla` branch in `data_loader.py`. We provide some script for you to begin with. You need to compose them into a transform object which is passed to `CustomDataset`.

1.2. Discriminator

The discriminator in this DCGAN is a convolutional neural network with the following architecture:



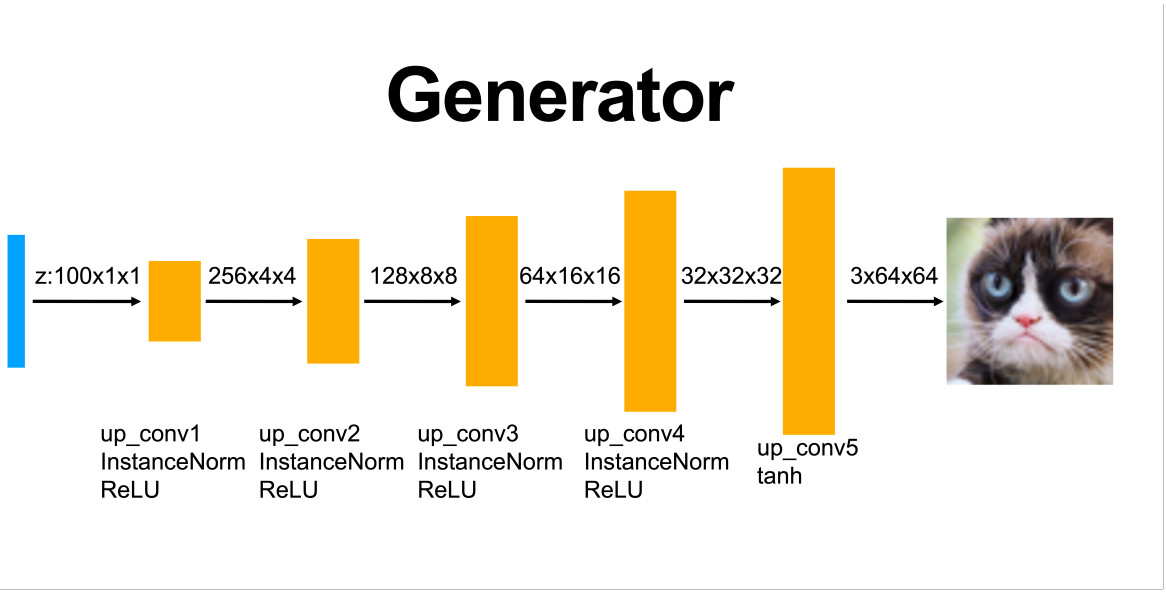
Task 1.2a. Padding: In the convolutional layers $1-4$ shown above, we downsample the spatial dimension of the input volume by a factor of 2. Given that we use kernel size $K = 4$ and stride $S = 2$, what should the padding be? Include the derivation in your writeup.

Task 1.2b. Implement this architecture by filling in the `__init__` and `forward` method of the `DCDiscriminator` class in `models.py`.

Note: The function `conv` in `models.py` has an optional argument norm: if norm is none, then conv simply returns a `torch.nn.Conv2d` layer; if `norm` is `instance/batch`, then `conv` returns a network block that consists of a `Conv2d` layer followed by a `torch.nn.InstanceNorm2d/BatchNorm2d` layer. Use the `conv` function in your implementation.

1.3. Generator

Now, we will implement the generator of the DCGAN, which consists of a sequence of upsample+convolutional layers that progressively upsample the input noise sample to generate a fake image. The generator in this DCGAN has the following architecture:



Task 1.3. implement this architecture by filling in the `__init__` and `forward` methods of the `DCGenerator` class in `models.py`.

Note: Use the `up_conv` function, analogous to the `conv` function used for the discriminator above, in your generator implementation. **We find that for the first layer, `up_conv1`, it is better to directly apply a convolution layer without any upsampling to get a `4x4` output. To do so, you'll need to think about what the kernel and padding size should be in this case. Feel free to use `up_conv` for the rest of the layers.**

1.4. Training Loop

Next, you will implement the training loop for the DCGAN. A DCGAN is simply a GAN with a specific type of generator and discriminator; thus, we train it in exactly the same way as a standard GAN. The pseudo-code for the training procedure is shown below.

Task 1.4. Implement the DCGAN training loop in `vanilla_gan.py`. The training loop alternates between updating the discriminator to distinguish real and generated images, and updating the generator to produce images that fool the discriminator. Follow the pseudocode below as a guide for the implementation.

You will notice that the training loop already calls `prepare_images(images, opts)` before passing images to the discriminator — this is the hook you will fill in during Task 1.5. For now, simply use it as written and implement the rest of the loop around it.

Algorithm 1 GAN Training Loop Pseudocode

procedure TRAINGAN
  Draw m training examples $\{x^{(1)}, \ldots, x^{(m)}\}$ from the data distribution p_{data}
  Draw m noise samples $\{z^{(1)}, \ldots, z^{(m)}\}$ from the noise distribution p_z
  Generate fake images from the noise: $G(z^{(i)})$ for $i \in \{1, \ldots, m\}$
  Compute the (least-squares) discriminator loss:

$$J^{(D)} = \frac{1}{2m} \sum_{i=1}^{m} \left[\Big(D(x^{(i)}) - 1 \Big)^2 \right] + \frac{1}{2m} \sum_{i=1}^{m} \left[\Big(D(G(z^{(i)})) \Big)^2 \right]$$

  Update the parameters of the discriminator
  Draw m new noise samples $\{z^{(1)}, \ldots, z^{(m)}\}$ from the noise distribution p_z
  Generate fake images from the noise: $G(z^{(i)})$ for $i \in \{1, \ldots, m\}$
  Compute the (least-squares) generator loss:

$$J^{(G)} = \frac{1}{m} \sum_{i=1}^{m} \left[\Big(D(G(z^{(i)})) - 1 \Big)^2 \right]$$

  Update the parameters of the generator
end procedure

1.5. Differentiable Augmentation

In this section, you will explore a technique that was not covered in class: *Differentiable Augmentation (DiffAugment)* [3]. You are encouraged to read the relevant part of the paper and understand the motivation behind the method before implementing it.

To further improve the data efficiency of GANs, we can apply *Differentiable Augmentation (DiffAugment)* [3]. Similarly to the data augmentation scheme from Task 1.1, the goal is to reduce discriminator overfitting when training on limited data. However, unlike standard dataset augmentation applied in the dataloader, DiffAugment is applied directly inside the training loop to both real and fake images before they are passed to the discriminator.

Importantly, the augmentations are differentiable, allowing gradients to flow through the augmented fake images back into the generator during training.

The implementation of DiffAugment itself is provided in `diff_augment.py`. Your job is to complete the `prepare_images` helper in `vanilla_gan.py`. This helper is called before images are passed to the discriminator.

Task 1.5. Complete `prepare_images(images, opts)` in `vanilla_gan.py`. When differentiable augmentation is enabled through `--use_diffaug`, the images should be processed using the provided DiffAugment implementation. Otherwise, they should be returned unchanged.

Note: `policy ='color,translation,cutout'` is already defined at the top of vanilla_gan.py. Use it when calling DiffAugment.

1.6. W&B Logging

Now that the training loop and DiffAugment integration are complete and frozen, you will add experiment tracking on top.

Task 1.6. Add W&B logging to `vanilla_gan.py` by filling in all **TODO 1.6** markers.

(a) Initialize a run. Use `wandb.init` to initialize a run at the start of `training_loop`, after the models, optimizers, and fixed noise are created.

- Use the project name `assignment1-dcgan`.
- Set the run name to include the value of `opts.use_diffaug` so that runs with and without DiffAugment can be distinguished easily in the W&B dashboard.
- Pass `config=vars(opts)`. This automatically records all command-line arguments and hyperparameters associated with the run.

(b) Log scalar losses. During training, log the discriminator losses:

- `D/real_loss`
- `D/fake_loss`
- `D/total_loss`

and the generator loss:

- `G/loss`

every `opts.log_step` iterations using `wandb.log`.

(c) Log the generated image grid to W&B inside `save_samples` every `opts.sample_every` iterations (use `wandb.log`, `wandb.Image`).

(d) Finish the run at the end of training with `wandb.finish()`.

1.7. Experiments

Task 1.7. Train the DCGAN by running the relevant experiment cells in the notebook, both with and without differentiable augmentation (`--use_diffaug`).

After completing the experiments, create a W&B Report summarizing your results. Your report should include:

- All loss curves for both runs
- Generated image grids both runs

In Addition, use the report to discuss the following:

- How the generator and discriminator losses evolve during training, and whether they match the expected behavior of a well-trained GAN
- How the visual quality of generated samples changes over time (for example, comparing early and late training iterations)
- The differences between standard dataset augmentation from Task 1.1 and DiffAugment from Task 1.5, including where each is applied, whether gradients flow through it, and their effect on training stability and output quality

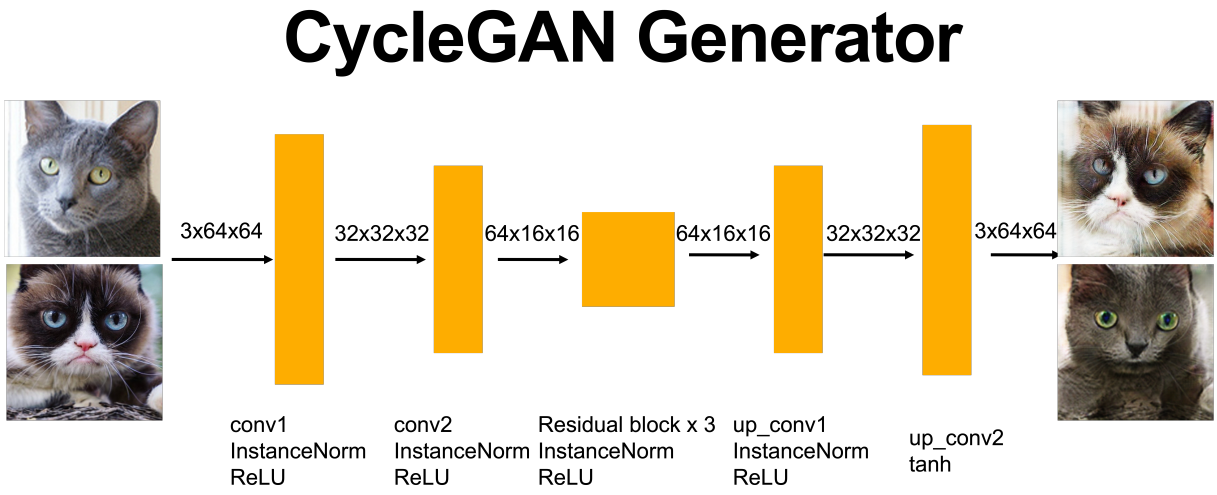
Before submitting, make sure the report visibility is set to *Anyone with a link can view*, and include the report link in your writeup.

2. Part 2: CycleGAN

You will implement the CycleGAN architecture for unpaired image-to-image translation, learning mappings between domain X (Grumpy cats) and domain Y (Russian Blue cats) without paired training data.

2.1. Generator

The generator in the CycleGAN has layers that implement three stages of computation: 1) the first stage encodes the input via a series of convolutional layers that extract the image features; 2) the second stage then transforms the features by passing them through one or more residual blocks; and 3) the third stage decodes the transformed features using a series of upsample+convolutional layers, to build an output image of the same size as the input. The residual block used in the transformation stage consists of a convolutional layer, where the input is added to the output of the convolution. This is done so that the characteristics of the output image (e.g., the shapes of objects) do not differ too much from the input.



Task 2.1. Implement the following generator architecture by completing the `__init__` and `forward` methods of the `CycleGenerator` class in `models.py`. To do this, you will need to use the `conv` and `up_conv` functions, as well as the `ResnetBlock` class, all provided in `models.py`.

Note: There are two generators in the `CycleGAN` model, $G_{X \to Y}$ and $G_{Y \to X}$, but their implementations are identical. Thus, in the code, $G_{X \to Y}$ and $G_{Y \to X}$ are simply different instantiations of the same class.

2.2. PatchDiscriminator

CycleGAN adopts a patch-based discriminator. Instead of directly classifying an image to be real or fake, it classifies the patches of the images, allowing CycleGAN to model local structures better. To achieve this effect, you will want the discriminator to produce spatial outputs (e.g., 4x4) instead of a scalar (1x1).

Task 2.2. Implement the `PatchDiscriminator` class in `models.py`. This can be done by slightly modifying the `DCDiscriminator` class.

(Hint: You can implement a PatchDiscriminator essentially by removing a layer from the DCDiscriminator.)

2.3. CycleGAN Training Loop

Finally, we will implement the CycleGAN training procedure, which is more involved than the procedure in Part 1. Similarly to Part 1, this training loop is not as difficult to implement as it may seem. There is a lot of symmetry in the training procedure, because all operations are done for both X $\to$ Y and Y $\to$ X directions.

Algorithm 2 CycleGAN Training Loop Pseudocode

procedure TRAINCYCLEGAN
  Draw a minibatch of samples $\{x^{(1)}, \ldots, x^{(m)}\}$ from domain X
  Draw a minibatch of samples $\{y^{(1)}, \ldots, y^{(m)}\}$ from domain Y
  Compute the discriminator loss on real images:

$$\mathcal{J}_{\text{real}}^{(D)} = \frac{1}{m}\sum_{i=1}^{m}\left(D_X(x^{(i)}) - 1\right)^2 + \frac{1}{n}\sum_{j=1}^{n}\left(D_Y(y^{(j)}) - 1\right)^2$$

  Compute the discriminator loss on fake images:

$$\mathcal{J}_{\text{fake}}^{(D)} = \frac{1}{m}\sum_{i=1}^{m}\left(D_Y(G_{X\to Y}(x^{(i)}))\right)^2 + \frac{1}{n}\sum_{j=1}^{n}\left(D_X(G_{Y\to X}(y^{(j)}))\right)^2$$

  Update the discriminators
  Compute the $Y \to X$ generator loss:

$$\mathcal{J}^{(G_{Y\to X})} = \frac{1}{n}\sum_{j=1}^{n}\left(D_X(G_{Y\to X}(y^{(j)})) - 1\right)^2 + \mathcal{J}_{\text{cycle}}^{(Y\to X\to Y)}$$

  Compute the $X \to Y$ generator loss:

$$\mathcal{J}^{(G_{X\to Y})} = \frac{1}{m}\sum_{i=1}^{m}\left(D_Y(G_{X\to Y}(x^{(i)})) - 1\right)^2 + \mathcal{J}_{\text{cycle}}^{(X\to Y\to X)}$$

  Update the generators
end procedure

Task 2.3. Complete the `training_loop` function in `cycle_gan.py`. There are 5 bullet points in the code for training the discriminators, and 6 bullet points in total for training the generators. Due to the symmetry between domains, several parts of the code you fill in will be identical except for swapping X and Y ; this is normal and expected.

2.4. Cycle Consistency Loss

The most interesting idea behind CycleGANs (and the one from which they get their name) is the idea of introducing a cycle consistency loss to constrain the model. The idea is that when we translate an image from domain X to domain Y, and then translate the generated image back to domain X, the result should look like the original image that we started with. The cycle consistency component of the loss is the mean error between the input images and their reconstructions obtained by passing through both generators in sequence (i.e., from domain X to Y via the $X \to Y$ generator, and then from domain Y back to X via the $Y \to X$ generator). The cycle consistency loss for the $Y \to X \to Y$ cycle is expressed as follows:

$$\frac{1}{m}\sum_{i=1}^{m}\left\|y^{(i)} - G_{X\to Y}\left(G_{Y\to X}(y^{(i)})\right)\right\|_1$$

The loss for the $X \to Y \to X$ cycle is analogous.

Task 2.4. Implement the cycle consistency loss in both directions in `cycle_gan.py`. There are two symmetric `TODO` sections, identical in structure except for swapping X and Y.

2.5. W&B Logging

Now add W&B logging to the CycleGAN training code.

Task 2.5. Add W&B logging to `cycle_gan.py` by filling in all **TODO 2.5** markers.

(a) Initialize a run. Use `wandb.init` to initialize a run at the start of `training_loop`, after the models and optimizers are created.

- Use the project name `assignment1-cyclegan`.
- Set the run name to include the values of `opts.X`, `opts.Y`, and `opts.use_cycle_consistency_loss` so that different datasets and loss settings can be distinguished easily in the W&B dashboard.
- Pass `config=vars(opts)` to `wandb.init`. This automatically records all command-line arguments and hyperparameters associated with the run.

(b) Log scalar losses. During training, log the discriminator losses:

- `D/real_loss`
- `D/fake_loss`
- `D/total_loss`

the generator losses:

- `G/xy_loss`
- `G/yx_loss`
- `G/total_loss`

and, when cycle consistency is enabled, the cycle consistency losses:

- `cycle/xyx_loss`
- `cycle/yxy_loss`

every `opts.log_step` iterations using `wandb.log`.

(c) Log the generated image grid inside `save_samples` every `opts.sample_every` iterations using `wandb.Image`. Log both translation directions:

- `samples/X_to_Y`
- `samples/Y_to_X`

(d) Finish the run at the end of training using `wandb.finish()`.

2.6. Experiments

Task 2.6.

Train the CycleGAN by running the relevant experiment cells in the notebook.

Run experiments with and without cycle consistency on the cat dataset. First run each setting for 1,000 iterations as a sanity check, and then train the cycle-consistency version for 10,000 iterations.

After completing the experiments, create a W&B Report summarizing your CycleGAN results. Your report should include:

- Relevant loss curves for all runs
- Generated image grids for all runs (both translation directions, $X \to Y$ and $Y \to X$)

In addition, use the report to discuss the following:

- Whether the 1,000-iteration runs behave as expected as a sanity check
- How the 10,000-iteration results improve qualitatively over the 1,000-iteration results
- Whether cycle consistency improves the outputs, grounding your explanation in the CycleGAN objective
- At least one specific example from the generated image grids of the 1,000-iteration runs where the effect of cycle consistency is noticable

Before submitting, make sure the report visibility is set to *Anyone with a link can view*, and include the report link in your writeup.

Task 2.7 (Bonus) [10 pts].

This bonus task is optional.

Choose **one** of the following options:

- **Method improvement:** Propose and implement an improvement or modification to the CycleGAN pipeline. Examples may include architectural changes, additional augmentations, alternative losses, different normalization schemes, training stabilisation techniques, or other ideas motivated by the literature.
- **Custom dataset:** Train the CycleGAN on your own image-to-image translation dataset. Your dataset should contain two visual domains suitable for unpaired translation.

Create a W&B Report documenting your experiments and results. Your report should clearly describe:

- What you changed or implemented
- Why you expected it to help
- Relevant loss curves and generated image samples
- A discussion of the qualitative results

Before submitting, make sure the report visibility is set to *Anyone with a link can view*, and include the report link in your writeup.

Submission

Submit a single `.zip` file via the course portal containing:

- `models.py`
- `vanilla_gan.py`
- `cycle_gan.py`
- `data_loader.py`
- Your PDF writeup

The writeup should include:

- Padding derivation (Task 1.2a)
- A link to your DCGAN W&B Report (Task 1.7)
- A link to your CycleGAN W&B Report (Task 2.6)
- A link to your bonus W&B Report, if you completed the bonus task (Task 2.7)

References

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Commun. ACM*, 63(11):139–144, October 2020.

[2] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks, 2016.

[3] Shengyu Zhao, Zhijian Liu, Ji Lin, Jun-Yan Zhu, and Song Han. Differentiable augmentation for data-efficient gan training, 2020.

[4] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2242–2251, 2017.